PACMAN Zipline Firmware and Software (3.X)

Branches, builds, boards and status

Maintained by the PACMAN agents (augie)

2026-11-04

1 Purpose and scope

Zipline (3.X) is the current PACMAN firmware and software, for the 1v5 board (also built for the 1v4), after the legacy (1.X) and hijinks (2.X) generations (companion notes `legacy.tex`, `hijinks.tex`). This note records where zipline stands: the branches, what was built from which commit, which build is on which board, the results, and what is open. The firmware itself (registers, units, timing) is described in `firmware.tex`; the checkout tools in `qaqc.tex`. Digests are SHA-256, first eight hex digits.

2 Repositories

Repository	Branch	State
`mulhearn/pacman-firmware`	`zipline-2023.2`	tag `v3.6` (a85681c, the release); branch at aaa46c1 (bump to 3.7, candidate), also `working-on-3.7-zipline-2023.2`
`mulhearn/pacman-software`	`zipline-2023.2`	tags `v3.6.0` (94e3f01), `v3.6.1` (90d4e19), `v3.6.2` (2c079c8) and `v3.6.3` (e9b21cf); branch at 54fee11 (bump to 3.6.4, candidate), also `working-on-v3.6.4-zipline-2023.2`

- Toolchain: Vivado 2023.2 (firmware), PetaLinux 2023.2 (images and the Linux applications), Vitis 2023.2 (bare metal).
- Firmware simulation: GHDL, `ghdl/` (test benches) and `ghdl/agents` (`make check` against `docs/requirements.txt`); `vivado/src/hdl` is kept in step with `ghdl/` in each commit.
- Software: the Linux applications in `petalinux/pkg/pacman-apps` (`pacman_cmdserver`, `pacman_dataserver`, `pacman_menu`, `pacman_hwcheck`, `pacman_fwcheck`, `pacman_lnxcheck`), one copy of the drivers (`drivers-common`) shared with the bare-metal menu (`apps/menu`).

3 Firmware 3.6

3.6 adds to 3.5: the ADC unit, the logic analyzer with ADC capture and cross triggers, the `_UPPER`/`_LOWER` register names, the RX packet timeout in units of 256 cycles, and the RX sweep that resumes at the next channel after a packet closed by its maximum. Released 2026-11-02 as 3.6 (a85681c, `BUILD_RELEASE`; tag `v3.6`); the next commit bumps the version to 3.7 (candidate).

Commit	Date	Build	Result
650dea6	2026-10-26	hw1v5	bit 82c2be63, BOOT.BIN c43b1340; timing met (WNS +0.115 ns, WHS +0.049 ns); `pacman_fwcheck` PASS (30 of 30 requirements, twice)
ae71967	2026-10-29	—	on hw1v4, a reset on tiles 1–8 asserts the single legacy reset (was 1–10)
928fcf0	2026-10-30	hw1v4	bit 2755aa4c; timing met (WNS +0.185 ns, WHS +0.011 ns)
a85681c	2026-11-02	hw1v5	the release, from a fresh clone: bit 6d84be5d, official (DIRTY 0); timing met (WNS +0.117 ns, WHS +0.014 ns), no critical warnings
a85681c	2026-11-02	hw1v4	the release: bit aa06b896, official; timing met (WNS +0.075 ns, WHS +0.025 ns); one critical warning (ADC inputs unconnected in the hw1v4 block design, open)

The release bitstreams are in the release images. Utilization of the release (hw1v5, post-place, xc7z020): 30 725 slice LUTs (57.8%), 50 712 registers (47.7%), 45 block RAM tiles (32.1%); hijinks 2.8 used 39 788 LUTs (74.8%), 60 644 registers (57.0%) and 99 tiles (70.7%), the v1.5 1.X line (`sync-fix`) 40 777 LUTs (76.7%). The release still holds a VIO (109 LUTs) and the Vivado debug hub it brings in (449 LUTs).

4 Releases

- A release is a dedicated commit that sets only the release flag (firmware `BUILD_RELEASE`, software `PACMAN_SOFTWARE_RELEASE` 1); the next commit bumps the version and clears the flag, nothing else. Builds of the release commit report "official", all others "candidate".
- Software 3.6.0 (94e3f01, 2026-11-03): Michael's production build, with the official site overlay, on the 1v5: `pacman_lnxcheck` 23 of 23, `pacman_fwcheck` 30 of 30, `pacman_hwcheck` all PASS, loopback 0 errors in 5.1 Mbit; LArPix v3 configured by larpix-lib through the servers (`larpix-lib.tex`).
- Software 3.6.1 (90d4e19, 2026-11-03): the ASIC power scripts (`power_up_tile.sh`, `power_down.sh`, `report_power.sh`) ported to zipline, through the command server at the `pacman_menu` nominal values; package scripts for the `pacman-apps` tarball (software-only updates, untarred at `/`) and the `firmware-dev` kit (the `BOOT.BIN` inputs and a `bootgen.bif`, for firmware developers).
- Software 3.6.2 candidate (1aef052, 2026-11-05), validated on the 1v5: `pacman-apps-v3_6_2-candidate.tar.gz` (b73a03ca, built in the cowork full project) untarred at `/`, reboot, servers from the init script. `pacman_lnxcheck -sd_write` 23 of 23, `pacman_fwcheck` 30 of 30 (servers stopped), `pacman_hwcheck` as for 3.6.0: read-only PASS, ADC and I2C stress PASS (8 of 8), power and loopback on tiles 6 and 7 (18 pass, 0 fail), ATC on tile 9 (11 pass, 0 fail); read-backs 4 of 4 and 64 of 64 (message version 1.1); `pacman_push` and `pacman_counter` on internal loopback, from felix and on the board: 420000 words each, 0 lost.
- Software 3.6.2 (2c079c8, release commit), Michael's production build in his release area: `pacman-apps-v3_6_2.tar.gz` cfffc412, root filesystem ac8ac3ef, `firmware-dev-v3_6_2` 8c0ab6b1 (byte-identical to v3_6_1); `BOOT.BIN` 801a5711, `boot.scr` 3b9b939d, `image.ub` 8094cb12 as for 3.6.0. Installed on the 1v5 (tarball at `/`, reboot): "PACMAN software 3.6.2 official"; read-backs 4 of 4 and 64 of 64. 3.6.1 retired.
- Software 3.6.3 candidate (294027b on `working-on-v3.6.3-zipline-2023.2`, after the bump 7906243), a `pacman_cmdserver` fix found through larpix-lib (2026-11-05): the main loop moved at most 100 TX buffers (one word per UART each) to the DMA per request handled, then blocked for the next request, so words beyond 100 per UART waited for a later request and their ASIC replies arrived late. Now the TX buffer drains between requests (`zmq_poll`, 1 ms while words remain), a word waits for buffer space (`tx_buffer_free`, at most 100 ms per request), word counts are 32 bits (wrapped above 2730 words) and the channel 16 bits (257 reached UART 0); every word not accepted

gets ERR 0xEEEE, the cause in the log: channels outside 1–40, no buffer space in time, requests too short, too large (over `MAX_WORDS`, 16000) or not the size their header gives (these crashed or stopped the server). The repeat and broadcast channels are removed; `tx_buffer_count` off by one fixed; the reset comment in `pacman_vspace.cc` corrected (0x00100410 internal, 0x00100420 full). On the 1v5 (the new server from `/tmp`, then the candidate tarball `pacman-apps-v3_6_3-candidate.tar.gz` 82fd5296 installed at `/` and rebooted): 256, 2000 and 16000 reads in one request exact, refusals answered with ERR, the server up after each; the TX timeout path not exercised.

- Software 3.6.3 (e9b21cf, release commit, 2026-11-07): the command server TX path above, data server threads pinned (main on cpu0, ZMQ I/O on cpu1, by inheritance), `pacman_zmqbench`, `pacman_boot` (address at boot in `last-boot.txt`), the logic analyzer and ADC capture setups in the menus, the test-DAC sawtooth in `pacman_hwcheck --adc` and `pacman_capture`. Michael's production build: `pacman-apps-v3_6_3.tar.gz` a162f6c6, root filesystem ee45ef7a, `firmware-dev-v3_6_3` 8c0ab6b1; the 3.6.0 images unchanged. Bundle `bundle-v3.6.0` with 3.6.3; migration notes with a single-chip section (tested on the 5x5, chip 11 alone).
- Point releases confined to `pacman-apps` ship only its tarball, no new images. The bundle (Michael's `bundle-v3.6.0`) is named for the firmware: the 3.6.0 images (`BOOT.BIN` 801a5711, `boot.scr` 3b9b939d, `image.ub` 8094cb12), the 3.6.1 root filesystem 48e50939, `pacman-apps-v3_6_1` 2a67b0a4, `firmware-dev-v3_6_1` 8c0ab6b1. The 3.6.1 images match 3.6.0 bit for bit except the root filesystem.

5 Software and images

Recent changes (pacman-software):

- 3d77617 to f516bc7 (2026-10-30): support for the ADC and logic analyzer; `pacman_hwcheck` as a test suite and `pacman_fwcheck`; the I2C overhaul (sticky status, bus recovery, MUX refusals as returns); one copy of the drivers; the I2C stress test.
- 2026-10-31: I2C at 100 kHz by default (640a124, needed on a hw1v4 card); ASIC TX at full drive in the configuration root (6d2ad8e); `--sma` tolerance 20% (41c6e00); the I2C device skip removed (3c9b2d0).
- 2026-11-01: the `pacman_hwcheck` tests as a submenu of both main menus (da113dc); one menu input on both platforms, a letter then Enter in every menu, a clean end at the end of input; one copy of the RX/TX and ADC menus for both platforms, the power menu folded into the I2C menu, a 16-BD ring for the menus (671a3dd); on the 1v4 since 2026-11-01.
- 2026-11-01 (290979d, which replaces 671a3dd with the changes below added): one RX/TX loopback core, which uses the configuration and ring as it finds them; in the RX/TX menu the ring size (16 to 512 BDs, default 512, set up by `m`), the loopback length, the RXTX UART mask (including tiles 2+3) and a BER check; `pacman_hwcheck --loopback` per tile, then all the masked tiles at once; the RX config and RX buffer config words built by functions. On the 1v4 since 2026-11-01.
- 2026-11-01 evening (4e9164c, 9a36407, 9e30354): `pacman_lnxcheck` memory limits 700–800 MB (1 GB less the DMA buffer); menu presets for the bench cards on tiles 6+7 and 9; the ramdisk and device tree loaded below the DMA buffer (U-Boot placed the ramdisk inside it, so the buffer's reservation failed); no PXE tries in the TFTP fall-through, and `boot.scr.uimg` copied into `images/linux`; the Python library on Python's path in every context (`pacman.pth`; `pacman_setup.sh` removed); root may log in over ssh with a password, once one is set; a site overlay: the untracked `petalinux/site` (files copied over the root filesystem, then an optional `post.sh` for edits such as the root password hash) applied by the in-place builds, with `petalinux/site.example` as the template. Tested on the 1v5.

The PetaLinux projects (`petalinux/ramdisk`, `petalinux/full`) are built in place, incrementally, after each software or firmware change:

Software	Date	Image	`BOOT.BIN`	`image.ub`
f516bc7	2026-10-30	ramdisk	41851385	7d452b37
3c9b2d0	2026-10-31	ramdisk	12813299	d7761858
3c9b2d0	2026-10-31	full	4c1d3a0e	8094cb12
671a3dd	2026-11-01	ramdisk	cec3182c	c4bd61b8
671a3dd	2026-11-01	full	96d3faca	8094cb12
290979d	2026-11-01	ramdisk	5bf59506	73368917
290979d	2026-11-01	full	348baf8a	8094cb12
9e30354	2026-11-01	ramdisk	31e01844	7b229452
9e30354	2026-11-01	full	8998d25c	8094cb12

6 Boards

Board	Firmware	Image	Results
1v5 (PACMAN 51 UCD)	a85681c	3.6.0 (SD)	Full SD image of the 3.6.0 release (root on p2), `pacman-apps` 3.6.1 installed by tarball. 2026-11-03, 3.6.1: ASIC configuration via `pacman_menu`; tile power and monitoring via `pacman_menu` and the power scripts (tile 1: VDDA 2365 mV, VDDD 1301 mV, matching the menu); SYNC words; `pacman_units`; RX/TX internal loopback from `pacman_cmdserver` to `pacman_dataserver` (`pacman_push`, `pacman_counter`, e.g. `-w 400 -n 100 -d 0`); RX test patterns at full speed on all 40 UARTs, 6060 packets/ms sustained (the theoretical maximum: 100 MHz / (10 clocks × 66 bits) × 40) with no loss, `pacman_dataserver` at 70% CPU. Earlier (650dea6, 290979d): 2026-10-30, f516bc7: `pacman_hwcheck`, every test PASS on the tiles carrying the test cards, except `--ana_mon` on tile 4 (a weak TESTP leg, set aside). 2026-11-01, 290979d: `pacman_lnxcheck` 23 of 23 PASS (memory limits 700–800 MB, 4e9164c), `pacman_fwcheck` 30 of 30 PASS, `pacman_hwcheck` PASS on tiles 6+7 (loopback card), 9 (ATC card) and the board-wide tests; the production SD boot with TFTP fall-through PASS; LArPix v3 on tile 1 replies and passes the ASIC loopback (`larpix.tex`). Bare-metal menu (4e9164c): boot, versions, I2C, hwcheck read-only, ADC, I2C stress and the ASIC hello PASS; the RX/TX and ASIC loopbacks lose words (open)
1v4	928fcf0	290979d (TFTP)	2026-11-01: the new menus work (letters and Enter, a clean end at the end of input, the hwcheck submenu: read only as expected, I2C stress PASS); the LArPix v3 on tile 1 does not reply (UART 0 streams junk; it replied exactly on 2026-10-31 with 3c9b2d0): the configuration sent right after the full reset is lost, and a 0.2 s pause fixes it. 2026-11-01, 290979d: a 16-BD ring set up from the menu, the ASIC hello exact twice, and the BER loopback on tiles 2+3 (internal loopback): 80000 of 80000 words, 0 bit errors in 5.12 Mbit

The 1v4 has no SD card: it boots by JTAG and TFTP; it is off the bench since 2026-11-01. On the 1v5, ssh and scp to the board work by key (sshd allows root by key only).

7 Open

- ZMQ, for 3.6.2 (Michael 2026-11-04; on `working-on-v3.6.2-zipline-2023.2`): the cause found and fixed. The data server's PUB socket (high-water mark 1, send timeout 0) dropped messages silently while `zmq_msg_send` succeeded: of four ASIC read replies in a row, the data server sent 4 (4 messages), felix received 2. High-water mark 1000 (6591987): read-backs 4 of 4 and 64 of 64; `pacman_push -w 40 -n 500 -d 1000` on internal loopback, 20000 sent, 20000 counted (with 1: 6680). Message version 1.1: the data message header carries `seq` (from 0, wraps) and `session` (random at each data server start, never 0); `pacman_counter` counts lost messages, restarts and unnumbered messages, prints its message count and a final report on exit, and closes oversize messages (ad1a58a; larpix-lib 0d43f7f). On the 1v5: 60000 words in 599 messages, 0 lost; a server restart during a run: 1 restart, 0 lost; a stand-in publisher with known gaps: all counted. `pacman_counter` and `pacman_push` take the PACMAN address (`-i`, default localhost; uncommitted, for review): run from felix against the 1v5, 60000 words in 600 messages, 0 lost.
- A stalled subscriber stops the data server (2026-11-04, the 1v5, tested): a felix SUB that connects and never reads, while `pacman_push -w 400 -n 1000 -d 0` runs on internal loopback: the data server aborts ("timeout waiting on msg_ready", its 1 s limit); `pacman_counter` on the board receives 5 messages (7600 words) of 400000. Without the stalled subscriber: 400000 words, 1000 messages, 0 lost. The data server hands ZMQ its own buffer and waits until the message is sent; one stalled peer holds it. Afterwards the RX path holds broken transfers (new data servers abort on the RX trailer assertions); only a reboot cleared it. Same in 3.6.1. By design (Michael 2026-11-04): no backpressure from ZMQ into the DMA, zero copy kept; a slow peer ends the run. Why a reboot was needed: the DMA reset (by the command server) resets only the DMA core; the RX buffer and the stream FIFO before the DMA are reset only at boot. A reset in the middle of a packet leaves its tail in the firmware, and the new ring's first transfer is that tail (not a whole number of words: the RX assertions). Fix (pacman.cc, for review): a transfer is used only if it is a whole packet (words, then a trailer counting them); otherwise it is discarded as an incomplete packet, with a warning, and counted (`pacman_rx_incomplete_packets`; the data server reports it). Tested twice: abort, restart of both servers, one 944-byte tail discarded, then read-backs 4 of 4 and 64 of 64; normal running (420000 words, 1500 messages): no discards.
- ZMQ throughput (2026-11-05, the 1v5, 3.6.2 official, RX test pattern on all 40 UARTs; UART losses from the firmware lost counters, zeroed first: they are 16-bit and saturate). Without a subscriber the data server receives the full input (3030.4 and 6060.9 packets/ms; at full speed the RX ring peaks at 510 of 512). With `pacman_counter`: quarter speed (1515) delivered whole, on the board and from felix, 0 lost, CPU 27–28% idle; half speed: 2589 per ms on the board (both cores full; 72 UART losses per ms), 2165 from felix (22% idle; 1237 UART losses per ms); full speed on the board: 2564 per ms, UART losses over 2621 per ms. No message lost (`seq`) in any run: the losses are at the UARTs, while the data server waits on ZMQ. Delivery tops out near 50–60 MB/s (2200–2600 packets/ms) whether the client is local or remote: the TCP send path, not the wire (refined 2026-11-06, below and in the summary).
- Delivery ceiling, by thread (2026-11-06, the 1v5, 3.6.3 candidate; `threads.sh`, `pubbench.py`, `subcount.py` in augie's `pacman/hwcheck/dev`). Half speed, a felix subscriber: 2350 packets/ms (56 MB/s); ZMQ's I/O thread at 100% system time on cpu0 (cpu0: 75% system, 24% softirq, 0% idle), the data server's main thread (ring polling and copy) 48% user on cpu1 (cpu1 47% idle). The limit is one thread: ZMQ's I/O thread in the kernel's TCP send, sharing its core with the network

softirq. A plain ZMQ PUB of prebuilt 64000-word messages (no DMA, no copy; zero copy, one message in flight as in the data server) to a felix subscriber: 3492 packets/ms (84 MB/s), the I/O thread again at 99% system time, now with the softirq mostly on the other core (cpu0 55% softirq, cpu1 75% system). Without the data server's work the ceiling is about 50% higher, still set by the TCP send; the gain may come from where the threads and the softirq run rather than from CPU time freed (cpu1 had 47% idle), not yet separated. Free-running sends (no wait) delivered 2596 packets/ms: the sending thread competes with the I/O thread.

- Core pinning (2026-11-06, the 1v5; `pin.sh`: `taskset` and the Ethernet IRQ's `smp_affinity`), half speed, a felix subscriber. Unpinned: 2307 packets/ms (55 MB/s), the I/O thread at 101% system time on the core that takes the network softirq, the other core 47% idle. The I/O thread alone on one core, the main thread and the IRQ on the other: 2600 (62 MB/s); mirrored: 2663 (64 MB/s); about 15% more. The I/O thread then runs at 82% (its core 16–17% idle); the other core is full (main thread about 40% user, softirq 52–54%). So the work is I/O thread (one core), softirq (half a core) and the data server's main thread (0.4 core), nearly both cores: with pinning the main thread's work is what limits, and removing it (the prebuilt-message benchmark, 3492) leaves the TCP send as the limit again.
- `pacman_zmqbench` (2026-11-06, for the 3.6.3 candidate, unstaged): publishes prebuilt data messages (64000 words, real header with `seq` and `session`) on 5556 with no DMA and no copy; `pacman_counter` counts them unchanged. Options `-w` (words per message), `-t` (seconds), `-q` (messages in flight, 1 as in the data server); refuses while the data server holds the port; ends the run when its subscriber leaves (with none, ZMQ drops messages and the send rate means nothing). On the 1v5: the counter on the board 3695 packets/ms (89 MB/s); on felix 2913–3637 (70–87 MB/s) over five runs, the Python benchmark 3417 on the same boot; 0 lost messages in every run, sessions matching.
- Core pinning in `pacman_zmqbench` and `pacman_dataserver` (2026-11-06, unstaged on the 3.6.3 candidate): ZMQ's I/O thread on cpu1 (`ZMQ_THREAD_AFFINITY_CPU_ADD`), the main thread on cpu0, where the Ethernet interrupts land (all of them in every run); the IRQ itself unchanged. `pacman_zmqbench` pinned by default (`-u` unpinned), with the interrupt split and a warning if they left cpu0. On the 1v5, to felix: `pacman_zmqbench` pinned 3617–3635 packets/ms (five runs), unpinned four near 3630 and one 2908. The data server at full speed (6060 in): pinned 2727–2737 (three runs, 65.5 MB/s), unpinned 2737, 2736, 2248; at half speed both deliver the full 3030. Pinned data server with the servers restarted in order: `hello_asic` 14 of 14, `dump_registers` 256 of 256.
- The data server's main thread, profiled (2026-11-06, the 1v5; a test build with timers in `pacman_poll_rx`, never committed; pinned, felix subscriber). Full speed (2699 packets/ms delivered, 64.8 MB/s): the main thread spends 99.5% of its wall time in `pacman_poll_rx`, 93% in its `memcpy` out of the DMA buffers, which runs at 69 MB/s; the rest of the RX handling 6%. Quarter speed (1515): `memcpy` at 161 MB/s, 23% of the time. The DMA buffers are mapped through `/dev/mem` with `O_SYNC`: uncached, so each word is a separate read from DRAM, slower still while the network interrupts share the core. At the ceiling the data server is the uncached copy, not the ring handling.
- `memcpy` in 16 KB blocks (one RX buffer) over 16 MB, the 1v5 idle (servers stopped; a scratch test, never committed): from the DMA region through `/dev/mem` with `O_SYNC` (as the drivers map it) 272 MB/s; without `O_SYNC` (cached) 426–438 MB/s; from `malloc` memory 412 MB/s; cached with the ARM `cacheflush` call on each block first 199–206 MB/s. (That call cleans rather than invalidates; a stand-in for the cost of software cache maintenance, not the exact operation.) So a cached copy is 1.6 times faster at idle, but software cache maintenance per buffer costs more than it saves; only hardware coherency (DMA through the ACP) gets the cached rate safely. Under load the uncached copy ran at 69 MB/s (full speed) and 161 MB/s (quarter speed), far below its idle 272: contention

for the memory bus and the core, not measured for a cached copy.

- Throughput limits, current (3.6.3, 2026-11-06, the 1v5, a remote client): the firmware and the data server alone sustain 6060 packets/ms (all 40 UARTs, the theoretical maximum). Delivery to a ZMQ client is CPU-limited: 2730 packets/ms (66 MB/s, about 18 UARTs at full speed) with the data server's threads pinned (3.6.3), the ZMQ I/O thread taking one core for the TCP send and the main thread sharing the other with the network interrupts; its uncached copy out of the DMA buffers is most of the main thread's time. The ZMQ/TCP path alone (`pacman_zmqbench`) reaches 3630 packets/ms (87 MB/s). Beyond the delivery limit, words are dropped at the UARTs and counted; no ZMQ message is lost.
- The VIO in the release (in discussion for 3.7).
- `pacman_menu` test patterns: the payload is written after the enable, so the first frame carries the old payload; `0x89ABDCDF` is probably meant as `0x89ABCDEF`.
- Bare metal: a small DMA ring (16 BDs) loses words in the loopback tests (Linux copes); the default 512 is fine.
- Planned for 3.7: registers for communication between processes (an I2C error flag, stalling the dataserver, lost-packet reports, the DMA race), polling for the end of a reset, more robust I2C (a termination option, a richer simulation), the `pattern.vhd` start-up.
- Robustness of the Vivado build: `create.tcl` adds the HDL from the repository, so HDL edited after the project was created partly enters a rebuild; release bitstreams are built from a fresh clone.